

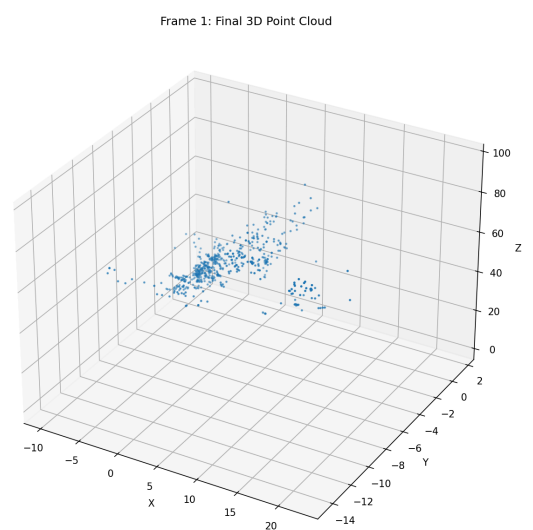
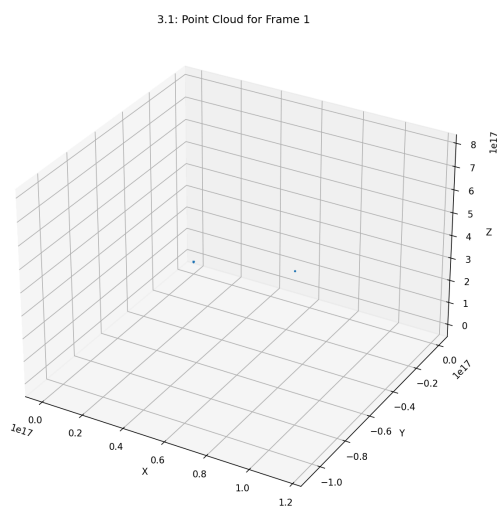
SLAM EX3

Ayala Houri 213867963, Nir Ellor Waizner 323816678

[Github](#)

3.1:

second image:



3.2:



3.3:

A) To transform a 3D point from the coordinate system of the first camera (left_0)

to the coordinate system of the second camera (left_1), we combine the 3x3 rotation matrix R (which describes the rotation from the left_0 camera frame to the left_1 camera frame) and the 3x1 translation vector t (which describes the displacement between the left_0 and left_1 camera frames) obtained from the PnP algorithm into a single 4x4 homogeneous transformation matrix, denoted as T.

The transformation matrix T is defined in block form as:

$$T = \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix}$$

Expressed explicitly component by component, the matrix structure is:

$$T = \begin{bmatrix} R_{11} & R_{12} & R_{13} & t_x \\ R_{21} & R_{22} & R_{23} & t_y \\ R_{31} & R_{32} & R_{33} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Using homogeneous coordinates, we append a scaling factor of 1 to our 3D point $P_{left0} = (X, Y, Z)^T$. Multiplying this vector by the matrix T yields the point's coordinates in the left_1 frame:

$$\begin{bmatrix} X' \\ Y' \\ Z' \\ 1 \end{bmatrix} = \begin{bmatrix} R_{11} & R_{12} & R_{13} & t_x \\ R_{21} & R_{22} & R_{23} & t_y \\ R_{31} & R_{32} & R_{33} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

This matrix multiplication efficiently computes the rigid body transformation:

$$\mathbf{P}_{left1} = \mathbf{R} * \mathbf{P}_{left0} + \mathbf{t}$$

B) To find the direct transformation from camera frame A to camera frame C, we can compose the two sequential transformations by substituting the expression for $T_{A \rightarrow B}(x)$ into $T_{B \rightarrow C}(y)$.

Let x be a 3D point in the coordinate system of camera A.

1. The coordinates of the point in frame B are given by (as we showed in A):

$$y = R1 * x + t1$$

2. The coordinates of the point in frame C are given by transforming its position from frame B:

$$z = R2 * y + t2$$

By substituting the expression for y into the second equation, we get:

$$T_{A \rightarrow C}(x) = R2 * (R1 * x + t1) + t2$$

Expanding the expression by distributing $R2$ yields the final transformation function:

$$\mathbf{T_{A \rightarrow C}(x) = (R2 * R1) * x + R2 * t1 + t2}$$

The extrinsic matrix of camera C:

The extrinsic matrix represents the combined rotation and translation required to map points from the reference frame (A) directly to the target frame (C).

From the derived transformation formula, we can isolate the effective rotation matrix and translation vector:

* Combined Rotation: $R2 * R1$

* Combined Translation: $R2 * t1 + t2$

Thus, the extrinsic matrix of camera C in $[R|t]$ block notation is:

$$[R_2 * R_1 \mid R_2 * t_1 + t_2]$$

C) The extrinsic matrix $[R|t]$ is a transformation matrix that maps 3D points from the global world coordinate system to the local camera coordinate system. By definition, the optical center (the physical lens) of the camera is always the exact origin, or $(0, 0, 0)$, of its own local camera coordinate system.

Therefore, following the provided hint, if we take the camera's global position vector in the world, which we can denote as $C_world = (x, y, z)^T$, and pass it through the extrinsic transformation matrix, the resulting output must be equal to zero. This is because the camera's position relative to itself is always zero.

We can write this geometric relationship as a linear equation:

$$R * C_world + t = 0$$

To find the actual location of the camera in the global world system, we will isolate the variable C_world :

1. Move the translation vector (t) to the right side of the equation:

$$R * C_world = -t$$

2. Multiply both sides by the inverse of the rotation matrix, denoted as

$\text{Inverse}(R)$:

$$C_world = -\text{Inverse}(R) * t$$

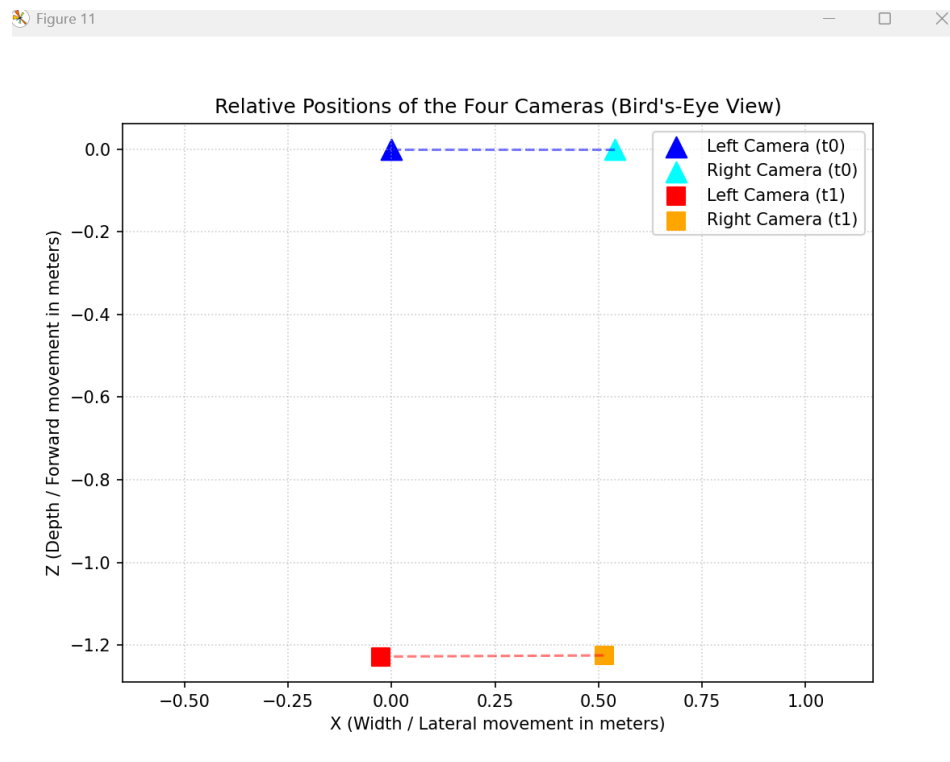
3. In computer vision, a rotation matrix is always orthogonal hence the inverse of the matrix is exactly equal to their transpose.

Replacing $\text{Inverse}(R)$ with $\text{Transpose}(R)$ gives us the final position formula:

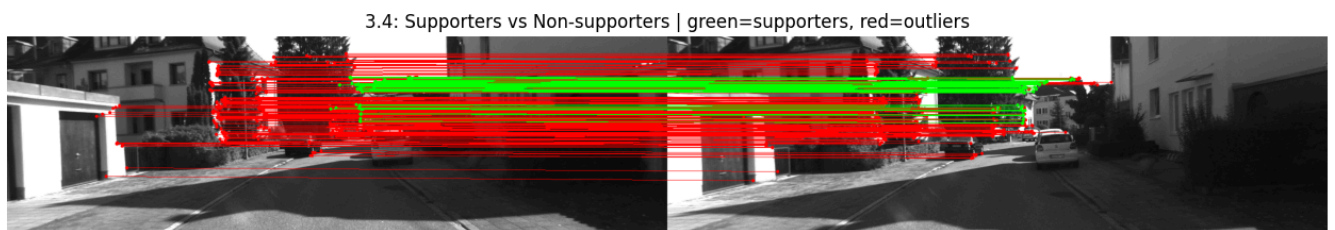
$$C_world = -\text{Transpose}(R) * t$$

=> The location of the camera in the global coordinate system is given by the formula: **$C_world = -\text{Transpose}(R) * t$**

D)

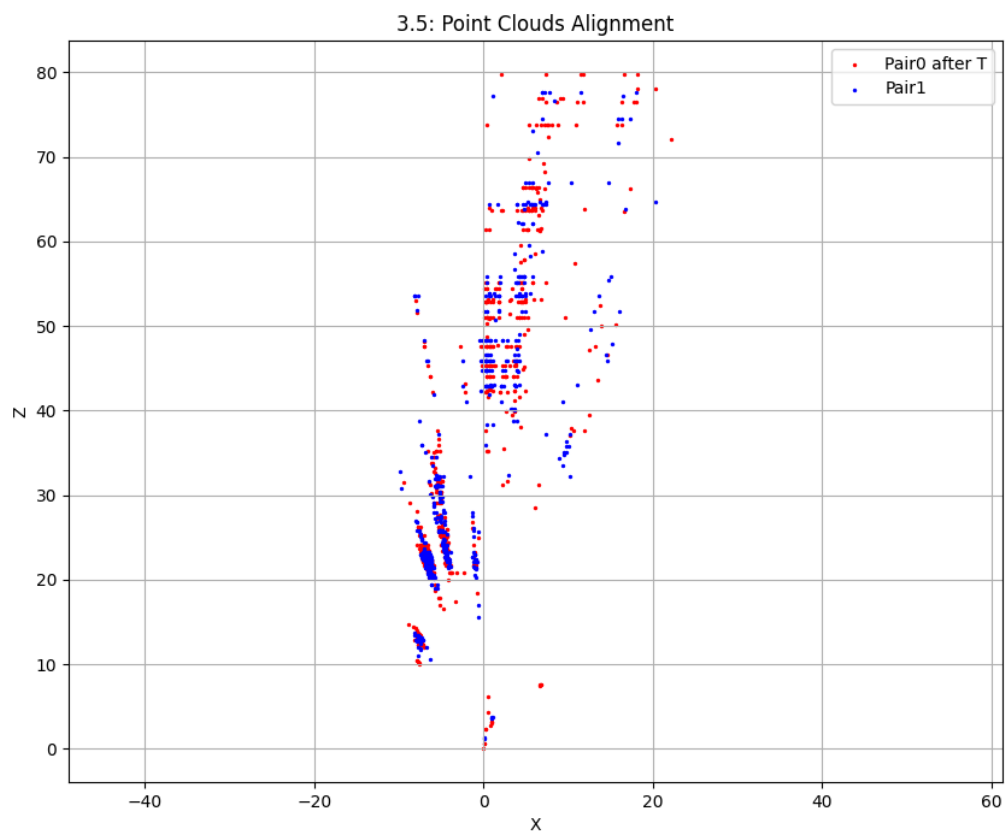
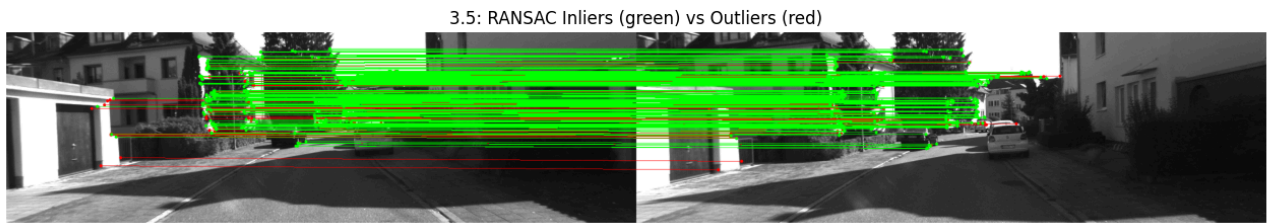


3.4:



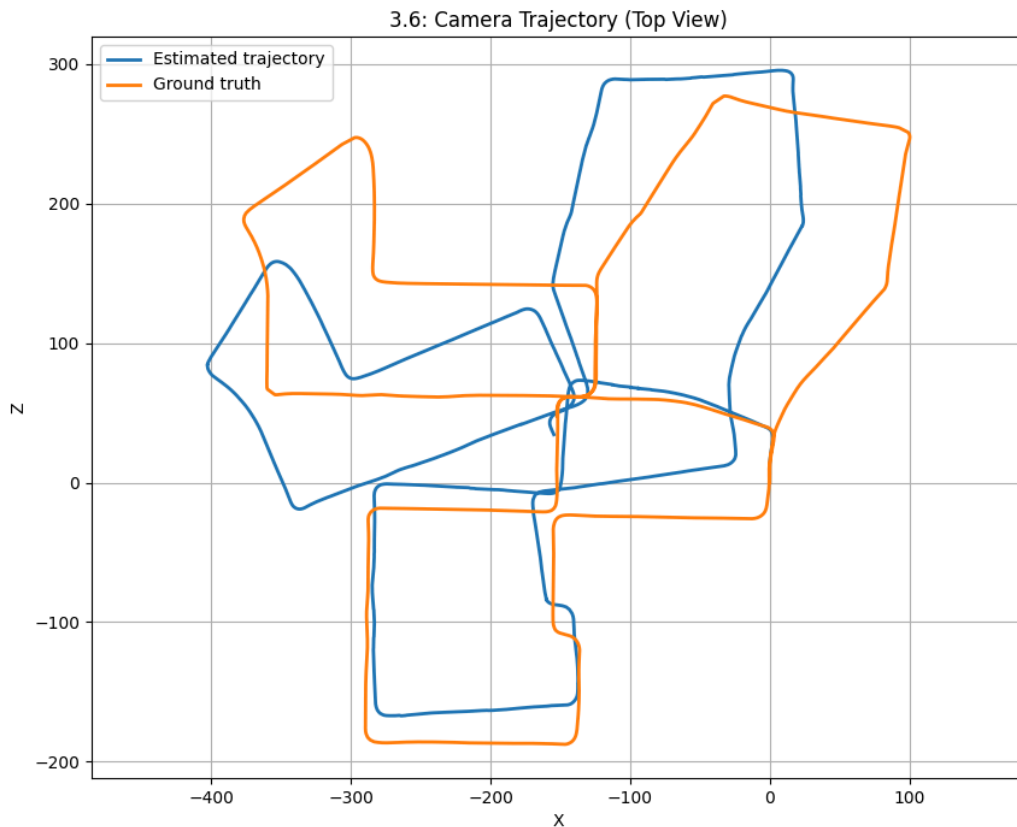
The reprojection consistency test identified 65 supporters and 297 non-supporters. The supporters (green matches) exhibit geometrically consistent projections across all four images, indicating that both the triangulated 3D points and the estimated camera transformation are reliable for these correspondences. Most non-supporters are likely caused by temporal mismatches, noisy triangulation, or pose estimation errors resulting from using only four points in the PnP stage without RANSAC.

3.5:



The custom RANSAC framework significantly improved the pose estimation results compared to the initial 4-point PnP solution. Out of 362 total correspondences, the refined model identified 291 inliers and only 71 outliers. The transformed point cloud from pair 0 aligns closely with the point cloud of pair 1, indicating that the estimated transformation successfully captures the relative camera motion between the two stereo pairs.

3.6:



The plot compares the estimated camera trajectory obtained from our stereo visual odometry pipeline (blue) with the KITTI ground-truth trajectory (orange), viewed from above in the coordinate system of the first left camera (left_0). The estimated trajectory successfully captures the overall structure and major turns of the route, indicating that the temporal tracking, triangulation, and PnP-RANSAC pipeline consistently recovered the vehicle motion across the sequence.

However, noticeable drift gradually accumulates over time, especially in the lateral direction and during long turns. This behavior is expected in a pure visual odometry system that estimates motion incrementally by chaining relative transformations without applying global optimization methods such as Bundle Adjustment, Loop Closure, or Pose Graph Optimization. Small rotational and translation errors therefore accumulate across thousands of frames.

The full tracking process over the entire sequence (~3300 frames) completed in 205.46 seconds, demonstrating that the optimized implementation — including frame reuse, dynamic RANSAC stopping, reduced feature count, and vectorized reprojection evaluation — achieved efficient large-scale tracking performance.